10. Automating Application Deployment Using a CI/CD Pipeline

Go to https://killercoda.com/docker/scenario/container-build and perform the following tasks:

Simulate a Basic CI/CD Pipeline Locally (In PWD)

Let's automate these steps:

1. **Build a Docker image**
2. **Run tests (if any)**
3. **Push to a local "registry" or deploy container**
4. **Restart the container**

🔨 1. Sample Project Structure

```
myapp/
├── app.py
├── Dockerfile
├── requirements.txt
└── deploy.sh
```

🐳 Dockerfile (vi Dockerfile)

```
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python3", "app.py"]
```

📄 app.py (vi app.py)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from CI/CD automated container!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

📦 requirements.txt (vi requirements.txt)

```
flask
```

⚙️ deploy.sh (CI/CD automation) (vi deploy.sh)

```
#!/bin/bash

PORT=8080

echo "Building Docker image..."
docker build -t myapp:latest .

echo "Stopping old containers..."
docker stop myapp || true
```

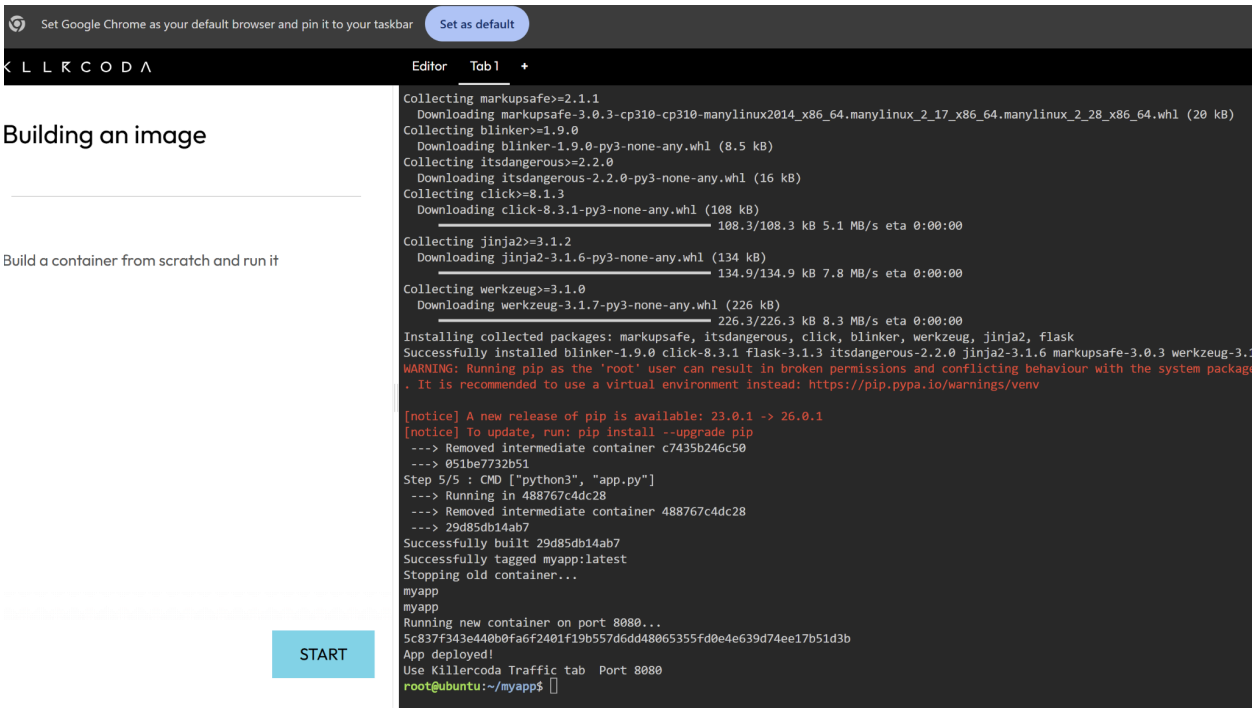
```
docker rm myapp || true

echo "Running new container on port $PORT..."
docker run -d --name myapp -p $PORT:8080 myapp:latest

echo "App deployed!"
echo "Use Killercoda Traffic tab Port $PORT"
```

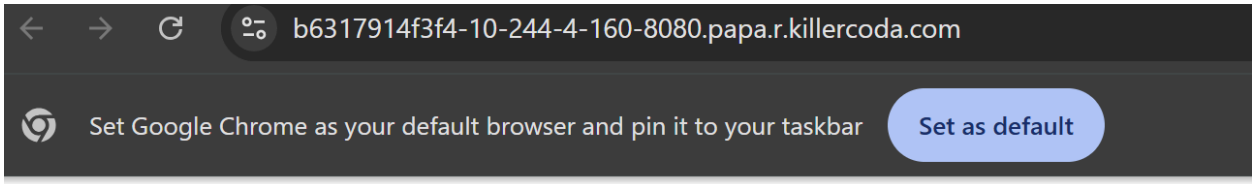
Make it executable:

```
chmod +x deploy.sh
```

Run:

```
./deploy.sh
```





Hello from CI/CD automated container!

****

//older version' s deploy.sh

```
#!/bin/bash

echo " Building Docker image..."
docker build -t myapp:latest .

echo " Stopping old container (if running)..."
docker stop myapp || true
docker rm myapp || true

echo " Running new container..."
docker run -d --name myapp -p 5000:5000 myapp:latest

echo " App deployed at:"
echo "http://$(hostname -i):5000"
```

👀 Access the App

If you're in labs.play-with-docker:

- **You'll see a `5000` port badge (click it)**
- **Or open manually:**

```
http://ip-XXXX-XX-XX-XX.play-with-docker.com:5000
```

✅ Step-by-Step Troubleshooting

🔹 **1. List all containers, even stopped ones**

```
docker ps -a
```

You'll likely see your container there with a `STATUS` like `Exited (1)` or similar.

🔹 **2. Check logs for why it crashed**

```
docker logs <container_id>
```

This will show exactly why the container failed — could be:

- **Missing file**
- **Syntax error in Python**
- **Python not found**

- **Flask crash**

🔹 **3. Restart the container manually (optional)**

```
docker start <container_id>
```

🔹 **4. Or rebuild and re-run it cleanly**

```
docker build -t flask-app .
docker run -d -p 5000:5000 flask-app
```

OUTPUT:

```
#####################################################################
#                          WARNING!!!!                              #
# This is a sandbox environment. Using personal credentials        #
# is HIGHLY! discouraged. Any consequences of doing so are         #
# completely the user's responsibilites.                           #
#                                                                  #
# The PWD team.                                                    #
#####################################################################
[node1] (local) root@192.168.0.18 ~
$ mkdir myapp/
[node1] (local) root@192.168.0.18 ~
$ cd myapp/
[node1] (local) root@192.168.0.18 ~/myapp
$ vi app.py
```

```
[node1] (local) root@192.168.0.18 ~/myapp
$ vi Dockerfile
[node1] (local) root@192.168.0.18 ~/myapp
```

```
[node1] (local) root@192.168.0.18 ~/myapp
$ vi requirements.txt
```

```
[node1] (local) root@192.168.0.18 ~/myapp
$ vi deploy.sh
```

```
$ chmod +x deploy.sh
[node1] (local) root@192.168.0.18 ~/myapp
$ ./deploy.sh
[🐳] Building Docker image...
[+] Building 10.5s (9/9) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 144B                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.10-slim                                           1.5s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [internal] load build context                                                                             0.0s
 => => transferring context: 813B                                                                             0.0s
 => [1/4] FROM docker.io/library/python:3.10-slim@sha256:65c843653048a3ba22c8d5083a022f44aef774974f0f7f70c    4.8s
 => => resolve docker.io/library/python:3.10-slim@sha256:65c843653048a3ba22c8d5083a022f44aef774974f0f7f70c    0.0s
 => => sha256:d7c79db7d957a3cfbc9d3930da23f24b9d988753b0bcbee3185cb7a76657fee3 5.31kB / 5.31kB                0.0s
 => => sha256:8a628cdd7ccc83e90e5a95888fcb0ec24b991141176c515ad101f12d6433eb96 28.23MB / 28.23MB              0.6s
 => => sha256:c96cb1c893456d3b64ac93cbebba12262429b64eca53584506f09ae223d1efe8 3.51MB / 3.51MB                0.4s
 => => extracting sha256:29e6ce9bbfedce1cc78818c9d0632f7e6f06c35278be1c3b0d979cd2b6b6c26b                     0.0s
 => [2/4] WORKDIR /app                                                                                        0.0s
 => [3/4] COPY . .                                                                                            0.0s
 => [4/4] RUN pip install -r requirements.txt                                                                 3.8s
 => exporting to image                                                                                        0.2s
 => => exporting layers                                                                                       0.2s
 => => writing image sha256:72fb7666b5c448f05f717225c8981333907aceb05e3e8a9994639aa5ebe6dfed                  0.0s
 => => naming to docker.io/library/myapp:latest                                                               0.0s
[🚀] Stopping old container (if running)...
Error response from daemon: No such container: myapp
Error response from daemon: No such container: myapp
[📦] Running new container...
9c92a723bfa62213cd041af8b089ef0f799b771e2bc429fded2607498a95c4eb
[✅] App deployed at:
http://192.168.0.18:5000
[node1] (local) root@192.168.0.18 ~/myapp
$
```

Hello from CI/CD automated container!